

Funções em Python

Organizando o Código

Ana Júlia Lima



Desenvolvido para
Young 1

Ctrl Play Santa Mônica
Vila Velha, Brasil
Fevereiro 2026

Sumário

1	O que são funções?	2
2	Como definir uma função	2
2.1	Como nomear funções	2
2.2	Docstring	2
3	Como uma função é constituída	2
3.1	Parâmetros	3
3.2	Escopo	3
3.3	Retorno	3
4	Passando informações	3
4.1	O que são parâmetros?	3
5	Entrada e saída de dados	3
5.1	Argumentos posicionais	3
5.2	Argumentos nomeados	3
5.3	Valores padrão (default)	3
5.4	Retorno de valores	3
5.5	Número arbitrário de argumentos	4
6	Funções compostas	4
7	Funções recursivas	4
8	Erros Comuns com Funções	5
8.1	1. Esquecer de chamar a função	5
8.2	2. Esquecer os parênteses ao chamar	5
8.3	3. Confundir print com return	5
8.4	4. Número errado de argumentos	6
8.5	5. Esquecer o return	6
8.6	6. Variáveis fora do escopo	6
8.7	7. Recursão sem condição de parada	6
8.8	Resumo	6
9	Lista de Exercícios — Funções	7
10	Gabarito — Funções	8
11	Mini Projeto Final — Sistema de Cadastro com Menu	10
11.1	Como o projeto vai funcionar	10
11.2	Código completo do projeto	10
11.3	Como o projeto foi organizado	11
11.4	Estrutura usada para guardar os dados	12
11.5	Exemplo de execução	12
11.6	Buscando uma pessoa específica	13
11.7	Desafios extras	13

1 O que são funções?

Funções são blocos de código que realizam uma tarefa específica. Elas servem para:

- Organizar o código
- Evitar repetição
- Facilitar manutenção
- Reutilizar lógica

```
def saudacao():  
    print("Olá!")
```

```
saudacao()
```

```
# Saída:  
# Olá!
```

2 Como definir uma função

Usamos a palavra-chave `def`.

```
def nome_da_funcao():  
    # código
```

2.1 Como nomear funções

Boas práticas:

- usar letras minúsculas
- separar palavras com underline
- nome descritivo

Exemplo:

```
def calcular_media():  
    pass
```

2.2 Docstring

Docstring é um texto que explica o que a função faz.

```
def soma(a, b):  
    """  
    Soma dois números e retorna o resultado.  
    """  
    return a + b
```

3 Como uma função é constituída

Uma função pode ter:

- Parâmetros
- Escopo
- Retorno

3.1 Parâmetros

São valores que a função recebe.

3.2 Escopo

É o espaço onde a variável existe.

3.3 Retorno

É o valor que a função devolve usando `return`.

```
def soma(a, b):  
    resultado = a + b  
    return resultado  
  
print(soma(2, 3))  
# Saída: 5
```

4 Passando informações

4.1 O que são parâmetros?

Parâmetros são variáveis que recebem valores dentro da função.

```
def mostrar_nome(nome):  
    print("Nome:", nome)  
  
mostrar_nome("Ana")
```

5 Entrada e saída de dados

5.1 Argumentos posicionais

```
def soma(a, b):  
    print(a + b)  
  
soma(2, 3)  
# Saída: 5
```

5.2 Argumentos nomeados

```
soma(a=2, b=3)
```

5.3 Valores padrão (default)

```
def saudacao(nome="Aluno"):  
    print("Olá,", nome)  
  
saudacao()  
# Olá, Aluno
```

5.4 Retorno de valores

```
def dobro(x):  
    return x * 2  
  
print(dobro(5))  
# Saída: 10
```

5.5 Número arbitrário de argumentos

```
def somar_todos(*numeros):  
    soma = 0  
    for n in numeros:  
        soma += n  
    return soma  
  
print(somar_todos(1,2,3,4))  
# Saída: 10
```

6 Funções compostas

Funções podem chamar outras funções.

```
def dobro(x):  
    return x * 2  
  
def soma_dobro(a, b):  
    return dobro(a) + dobro(b)  
  
print(soma_dobro(2,3))  
# Saída: 10
```

7 Funções recursivas

Uma função recursiva chama a si mesma.

```
def contagem(n):  
  
    if n == 0:  
        return  
  
    print(n)  
    contagem(n-1)  
  
contagem(5)  
  
# Saída:  
# 5  
# 4  
# 3  
# 2  
# 1
```

Importante:

Toda função recursiva precisa de uma condição de parada.

Resumo Final

- Funções organizam e reutilizam código.
- Parâmetros recebem dados.
- Return devolve valores.
- Podemos usar argumentos posicionais, nomeados e padrão.
- Funções podem chamar outras funções.
- Recursão é quando uma função chama a si mesma.

8 Erros Comuns com Funções

Funções são extremamente importantes, mas alguns erros aparecem com frequência. Vamos ver os principais.

8.1 1. Esquecer de chamar a função

Definir uma função não faz ela executar automaticamente.

Exemplo:

```
def saudacao():  
    print("Olá!")
```

Nada acontece!

Correto:

```
saudacao()
```

```
# Saída:  
# Olá!
```

8.2 2. Esquecer os parênteses ao chamar

```
def saudacao():  
    print("Olá!")
```

```
print(saudacao)
```

Isso mostra a referência da função, não executa.

Correto:

```
print(saudacao())
```

8.3 3. Confundir print com return

Errado:

```
def soma(a, b):  
    print(a + b)
```

```
resultado = soma(2, 3)
```

```
print(resultado)
```

```
# Saída:  
# 5  
# None
```

Por quê?

- print mostra o valor
- return devolve o valor

Correto:

```
def soma(a, b):  
    return a + b
```

```
resultado = soma(2, 3)
```

```
print(resultado)
```

```
# Saída: 5
```

8.4 4. Número errado de argumentos

```
def soma(a, b):  
    return a + b  
  
soma(5)
```

Erro porque faltou um argumento.

8.5 5. Esquecer o return

```
def dobro(x):  
    x * 2  
  
print(dobro(5))  
# Saída: None
```

Correto:

```
def dobro(x):  
    return x * 2
```

8.6 6. Variáveis fora do escopo

```
def teste():  
    x = 10  
  
print(x)
```

Erro: a variável só existe dentro da função.

8.7 7. Recursão sem condição de parada

```
def infinito(n):  
    print(n)  
    infinito(n+1)
```

Isso gera erro de recursão infinita.

Sempre precisa de condição de parada!

8.8 Resumo

- Sempre chame a função com parênteses.
- Use return quando quiser devolver valores.
- Verifique quantidade de argumentos.
- Cuidado com escopo.
- Toda recursão precisa parar.

9 Lista de Exercícios — Funções

Nível 1 — Básico

1. Crie uma função chamada `mensagem()` que imprima "Olá, mundo!".
2. Crie uma função que receba um nome e mostre: "Olá, `nome`".
3. Crie uma função que receba um número e retorne o dobro dele.
4. Crie uma função que receba dois números e mostre a soma deles.

Nível 2 — Parâmetros e Retorno

1. Crie uma função que receba três números e retorne a média deles.
2. Crie uma função com valor padrão: Se nenhum nome for passado, mostre "Visitante".
3. Crie uma função que receba vários números (`*args`) e retorne a soma.
4. Crie uma função que receba um número e diga se ele é par ou ímpar.

Nível 3 — Funções compostas e recursivas

1. Crie uma função que calcule o quadrado de um número e outra que use essa função para somar dois quadrados.
2. Crie uma função que conte de um número até zero usando recursão.
3. Crie uma função que receba uma lista de números e retorne o maior valor.
4. Crie uma função que receba um número e retorne o fatorial usando recursão.

10 Gabarito — Funções

Nível 1

1.

```
def mensagem():  
    print("Olá, mundo!")  
  
mensagem()
```

2.

```
def saudacao(nome):  
    print("Olá,", nome)  
  
saudacao("Ana")
```

3.

```
def dobro(x):  
    return x * 2  
  
print(dobro(5))  
# Saída: 10
```

4.

```
def soma(a, b):  
    print(a + b)  
  
soma(2, 3)  
# Saída: 5
```

Nível 2

1.

```
def media(a, b, c):  
    return (a + b + c) / 3  
  
print(media(6, 7, 8))
```

2.

```
def saudacao(nome="Visitante"):  
    print("Olá,", nome)  
  
saudacao()
```

3.

```
def somar_todos(*numeros):  
    total = 0  
  
    for n in numeros:  
        total += n  
  
    return total  
  
print(somar_todos(1,2,3,4))
```

4.

```
def verificar(numero):  
  
    if numero % 2 == 0:  
        return "Par"  
    else:  
        return "Ímpar"  
  
print(verificar(4))
```

Nível 3

1.

```
def quadrado(x):  
    return x * x  
  
def soma_quadrados(a, b):  
    return quadrado(a) + quadrado(b)  
  
print(soma_quadrados(2,3))  
# Saída: 13
```

2.

```
def contagem(n):  
  
    if n == 0:  
        return  
  
    print(n)  
    contagem(n-1)  
  
contagem(5)
```

3.

```
def maior(lista):  
    return max(lista)  
  
print(maior([1,5,3,9,2]))
```

4.

```
def fatorial(n):  
  
    if n == 0:  
        return 1  
  
    return n * fatorial(n-1)  
  
print(fatorial(5))  
# Saída: 120
```

11 Mini Projeto Final — Sistema de Cadastro com Menu

A ideia será criar um sistema que:

- faça cadastro de uma pessoa pedindo nome, idade e cidade natal
- possa cadastrar mais de uma pessoa
- mostre todos os cadastros já feitos
- procure o cadastro de uma pessoa específica
- encerre o programa

11.1 Como o projeto vai funcionar

O programa mostrará um menu com opções.

A pessoa poderá escolher:

- 1 → cadastrar uma nova pessoa
- 2 → mostrar todos os cadastros
- 3 → buscar uma pessoa pelo nome
- 4 → sair

11.2 Código completo do projeto

```
def mostrar_menu():
    print("\n--- MENU ---")
    print("1 - Cadastrar pessoa")
    print("2 - Mostrar todos os cadastros")
    print("3 - Buscar pessoa pelo nome")
    print("4 - Sair")

def cadastrar_pessoa():
    nome = input("Digite o nome: ")
    idade = int(input("Digite a idade: "))
    cidade = input("Digite a cidade: ")

    cadastro = {
        "nome": nome,
        "idade": idade,
        "cidade": cidade
    }

    return cadastro

def mostrar_todos(cadastros):
    if len(cadastros) == 0:
        print("Nenhum cadastro encontrado.")
        return

    print("\n--- TODOS OS CADASTROS ---")

    for pessoa in cadastros:
        print("Nome:", pessoa["nome"])
        print("Idade:", pessoa["idade"])
        print("Cidade:", pessoa["cidade"])
        print("-----")
```

```

def buscar_pessoa(cadastros):
    if len(cadastros) == 0:
        print("Nenhum cadastro encontrado.")
        return

    nome_busca = input("Digite o nome da pessoa: ")

    for pessoa in cadastros:
        if pessoa["nome"].lower() == nome_busca.lower():
            print("\n--- CADASTRO ENCONTRADO ---")
            print("Nome:", pessoa["nome"])
            print("Idade:", pessoa["idade"])
            print("Cidade:", pessoa["cidade"])
            return

    print("Pessoa não encontrada.")

def main():
    cadastros = []

    while True:
        mostrar_menu()
        opcao = input("Escolha uma opção: ")

        if opcao == "1":
            pessoa = cadastrar_pessoa()
            cadastros.append(pessoa)
            print("Cadastro realizado com sucesso!")

        elif opcao == "2":
            mostrar_todos(cadastros)

        elif opcao == "3":
            buscar_pessoa(cadastros)

        elif opcao == "4":
            print("Encerrando o programa...")
            break

        else:
            print("Opção inválida. Tente novamente.")

main()

```

11.3 Como o projeto foi organizado

Cada função tem uma responsabilidade específica:

- `mostrar_menu()` → mostra as opções disponíveis
- `cadastrar_pessoa()` → pede os dados e cria um cadastro
- `mostrar_todos(cadastros)` → exibe todos os cadastros
- `buscar_pessoa(cadastros)` → procura uma pessoa pelo nome
- `main()` → controla o funcionamento geral do sistema

11.4 Estrutura usada para guardar os dados

Os dados foram organizados assim:

- cada pessoa é representada por um **dicionário**
- todos os cadastros ficam dentro de uma **lista**

Exemplo de como isso fica internamente:

```
cadastros = [  
    {"nome": "Ana", "idade": 18, "cidade": "Vila Velha"},  
    {"nome": "Carlos", "idade": 20, "cidade": "Vitória"}  
]
```

11.5 Exemplo de execução

```
--- MENU ---  
1 - Cadastrar pessoa  
2 - Mostrar todos os cadastros  
3 - Buscar pessoa pelo nome  
4 - Sair  
Escolha uma opção: 1  
Digite o nome: Ana  
Digite a idade: 18  
Digite a cidade: Vila Velha  
Cadastro realizado com sucesso!
```

```
--- MENU ---  
1 - Cadastrar pessoa  
2 - Mostrar todos os cadastros  
3 - Buscar pessoa pelo nome  
4 - Sair  
Escolha uma opção: 1  
Digite o nome: Carlos  
Digite a idade: 20  
Digite a cidade: Vitória  
Cadastro realizado com sucesso!
```

```
--- MENU ---  
1 - Cadastrar pessoa  
2 - Mostrar todos os cadastros  
3 - Buscar pessoa pelo nome  
4 - Sair  
Escolha uma opção: 2
```

```
--- TODOS OS CADASTROS ---  
Nome: Ana  
Idade: 18  
Cidade: Vila Velha  
-----  
Nome: Carlos  
Idade: 20  
Cidade: Vitória  
-----
```

11.6 Buscando uma pessoa específica

A busca é feita pelo nome.

Escolha uma opção: 3

Digite o nome da pessoa: Ana

--- CADASTRO ENCONTRADO ---

Nome: Ana

Idade: 18

Cidade: Vila Velha

11.7 Desafios extras

Depois de montar esse sistema, você pode tentar melhorá-lo:

- adicionar e-mail no cadastro
- mostrar se a pessoa é maior ou menor de idade
- permitir remover um cadastro
- permitir editar dados de uma pessoa já cadastrada
- mostrar quantas pessoas estão cadastradas